LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK

“LAPORAN AKHIR PEKAN 1-5”

disusun Oleh:

Muhammad Fedora Argadyaksa

2511533016

Dosen Pengampu:

Nurfiah, S.ST, M.Kom

Asisten Praktikum:

Siti Fadhilah Rahmi



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

1. Langkah Kerja Praktikum

a. Langkah pertama yaitu, buka aplikasi yang dibutuhkan untuk praktikum (Eclipse) Lalu buat package pekan 1-5 dan berisi class Rekening, Main, Transaksi yang menggunakan kode program pada pekan sebelumnya dan juga terdapat class baru yang diberikan pada modul ini.

b. Langkah kedua, membuat kode program yang terdapat penambahan atau perubahan di dalam modul praktikum :

A. Pekan 1

a. Class Rekening :

```
1. package Pekan_1;
2. 
3. public class Rekening {
4.         String nomorRekening;
5.         String namaPemilik;
6.         double saldo;
7. 
8.         public Rekening(String nomor, String nama, double saldoAwal) {
9.                 nomorRekening = nomor;
10.                namaPemilik = nama;
11.                saldo = saldoAwal;
12.                System.out.println("Rekening atas nama " + namaPemilik
   + " berhasil dibuat dengan saldo Rp " + saldo);
13.        }
14. 
15.        public void setorTunai(double nominal) {
16.                if (nominal > 0){
17.                        saldo += nominal;
18.                        System.out.println("Setor tunai Rp" + nominal +
   " berhasil. Saldo saat ini : Rp" + saldo);
19.                } else {
20.                        System.out.println("Gagal : Nominal setor harus
   lebih dari 0!");
21.                }
22.        }
23. 
24.        public void tarikTunai(double nominal) {
25.                if (nominal <= 10000 && nominal <= saldo) {
26.                        saldo -= nominal;
27.                        System.out.println("Tarik tunai Rp " + nominal +
   " berhasil. Saldo saat ini : Rp" + saldo);
```

```
28.             } else {
29.                   System.out.println("Transaksi Gagal: Saldo tidak
   mencukupi. Saldo Anda: Rp" + saldo);
30.             }
31.       }
32.
33.       public boolean cocokDenganNomor(String nomor) {
34.             if (this.nomorRekening.equals(nomor)) {
35.                   return true;
36.             } else {
37.                   return false;
38.             }
39.       }
40.
41.       public void cekInformasi() {
42.             System.out.println("--- INFO REKENING ---");
43.             System.out.println("No. Rekening : " + nomorRekening);
44.             System.out.println("Nama Pemilik : " + namaPemilik);
45.             System.out.println("Saldo Akhir  : " + saldo);
46.             System.out.println("-----------------------");
47.       }
48.
49. }
```

Kelas Rekening dalam package Pekan_1 berfungsi sebagai blueprint objek rekening bank, dengan tiga atribut: nomorRekening dan namaPemilik (String), serta saldo (double); constructor Rekening(String nomor, String nama, double saldoAwal) mengisi ketiga atribut tersebut saat objek dibuat sekaligus mencetak pesan konfirmasi. Method setorTunai(double nominal) menambah saldo jika nominal lebih besar dari 0, sedangkan tarikTunai(double nominal) mengurangi saldo hanya jika nominal ≤ 10000 dan tidak melebihi saldo yang tersedia — jika salah satu syarat gagal, transaksi ditolak. Method cocokDenganNomor(String nomor) mengembalikan true/false dengan membandingkan nomorRekening objek saat ini dengan nomor yang diinput, berguna untuk mencari rekening di dalam ArrayList. Terakhir, cekInformasi() menampilkan seluruh data rekening (nomor, nama, saldo) dalam format rapi. Singkatnya, kelas ini merepresentasikan objek rekening bank lengkap dengan kemampuan menyetor, menarik, mencocokkan nomor, dan menampilkan info dirinya sendiri.

b. Class Main

```java
package Pekan_1;

import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Rekening akunAktif = null; // Objek belum diinstalisasi(null)
        boolean isRunning = true;

        System.out.println("=== SISTEN PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\nMenu Utama : ");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");

            System.out.printf("Pilih Menu : ");

            int pilihan = input.nextInt();
            input.nextLine(); // Membersihkan buffer enter

            switch (pilihan) {
            case 1 :
                System.out.print("Masukkan No Rekening: ");
                String no = input.nextLine();
                System.out.print("Masukkan Nama Pemilik : ");
                String nama = input.nextLine();
                System.out.print("Masukkan Saldo Awal : ");
                double saldo = input.nextDouble();

                // Instansiasi Objek / Menjalankan Constructor
                akunAktif = new Rekening(no, nama, saldo);

                break;

            case 2 :
                if (akunAktif == null) {
                    System.out.println("Error : Mohon maaf, Anda belum memiliki nomor rekening!");
                } else {
                    System.out.print("Masukkan nominal setor : ");
                    double setor = input.nextDouble();
                    akunAktif.setorTunai(setor); // Memanggil Behavior / method
                }
                break;

            case 3 :
                System.out.println("Fitur akan dikerjakan secara mandiri");
                break;

            case 4 :
                if (akunAktif == null) {
                    System.out.println("Error : Anda belum membuka rekening!");
                } else {
                    akunAktif.cekInformasi();
                }
                break;

            case 0 :
                isRunning = false;
                System.out.println("Sistem ditutup. Terima Kasih!");
                break;

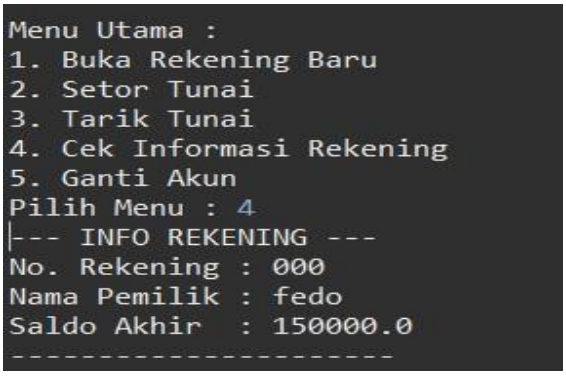
            default:
                System.out.println("Pilihan tidak valid!");
            }
        }
        input.close();
    }
}
```

Kelas Main ini adalah program utama sistem perbankan mini yang berjalan dalam perulangan while (isRunning) untuk terus menampilkan menu selama program belum ditutup, dengan akunAktif (bertipe Rekening, awalnya null) menyimpan rekening yang sedang aktif digunakan dan ArrayList rekeningAktif

menyimpan semua rekening yang pernah dibuat; pada case 1 user membuat rekening baru yang langsung dijadikan akunAktif dan ditambahkan ke ArrayList; case 2 dan case 3 masing-masing memproses setor dan tarik tunai terhadap akunAktif (dengan pengecekan saldo/null sebelum transaksi); case 4 menampilkan info akunAktif jika sudah ada; case 5 (Ganti Akun) meminta nomor rekening lalu mencarinya di dalam rekeningAktif menggunakan cocokDenganNomor() — jika cocok, objek tersebut dijadikan akunAktif baru dan flag ditemukan diset true, jika tidak ada yang cocok setelah loop selesai maka muncul pesan gagal; dan case 0 mengakhiri program dengan mengubah isRunning menjadi false. Singkatnya, program ini memungkinkan user membuat banyak rekening, berpindah rekening aktif, lalu melakukan transaksi terhadap rekening yang sedang aktif tersebut.

Output :





Tugas/Latihan Praktikum

A. Implementasi Fitur Tarik Tunai

- Tambahkan method tarikTunai(double nominal) ke dalam class Rekening.
- Aturan : Saldo tidak boleh menjadi minus dan minimal nominal Tarik tunai adalah 10.000. Jika pengguna mencoba menarik nominal yang lebih besar dari saldo saat ini atau nominal kecil dari 10.000, program tidak boleh crash/error, melainkan menampilkan peringatan ke konsol: "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp[nominal]" atau “Transaksi Gagal : Minimal nomonal penarikan 10.000”.
- Terapkan method ini dengan menu nomor 3 pada class Main**.**

a. Class Rekening

```
1.     public void tarikTunai(double nominal) {
2.     if (nominal <= 10000 && nominal <= saldo) {
3.     saldo -= nominal;
4.     System.out.println("Tarik tunai Rp " + nominal +
    " berhasil. Saldo saat ini : Rp" + saldo);
5.     } else {
6.     System.out.println("Transaksi Gagal: Saldo tidak mencukupi.
Saldo Anda: Rp" + saldo);
7.     }
8.     }
```

Method tarikTunai(double nominal) berfungsi untuk mengurangi saldo rekening dengan syarat ganda yang harus dipenuhi bersamaan (&&), yaitu nominal penarikan harus kurang dari atau sama dengan 10000 dan tidak boleh melebihi saldo yang tersedia — jika kedua syarat terpenuhi, saldo dikurangi (saldo -= nominal) dan dicetak pesan sukses beserta saldo terkini, namun jika salah satu syarat saja tidak terpenuhi (misalnya nominal lebih dari 10000 atau melebihi

saldo yang ada), transaksi ditolak dan dicetak pesan bahwa saldo tidak mencukupi beserta saldo yang tersisa.

b. Class Main

```
1.      case 3 :
2.      if (akunAktif.saldo < 10000) {
3.      System.out.println("Transaksi
                Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" +
                                                akunAktif.saldo);
4.      } else {
5.      System.out.print("Masukkan nominal tarik : ");
6.      double tarik = input.nextDouble();
7.      akunAktif.tarikTunai(tarik);
8.      }
9.      break;
```

Case 3 (Tarik Tunai) mengecek terlebih dahulu apakah saldo akunAktif kurang dari 10000 — jika ya, penarikan langsung ditolak dengan pesan saldo tidak mencukupi, namun jika saldo mencukupi maka user diminta memasukkan nominal tarik yang kemudian diproses lewat method tarikTunai() milik akunAktif; sedangkan case 5 (Ganti Akun) pertama-tama mengecek apakah rekeningAktif (ArrayList) masih kosong, jika tidak maka user diminta memasukkan nomor rekening yang dicari (cariNomor) dan flag ditemukan disiapkan bernilai false, lalu dilakukan perulangan for-each ke setiap objek r di dalam rekeningAktif sambil mencocokkan nomornya menggunakan r.cocokDenganNomor(cariNomor) — begitu ditemukan yang cocok, objek tersebut dijadikan akunAktif baru, ditemukan diubah menjadi true, pesan sukses dicetak, dan loop dihentikan dengan break, namun jika sampai akhir loop ditemukan

tetap false maka ditampilkan pesan bahwa nomor rekening tidak ditemukan.

Output :

```
=== SISTEN PERBANKAN MINI ===

Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
Pilih Menu : 1
Masukkan No Rekening: 000
Masukkan Nama Pemilik : fedo
Masukkan Saldo Awal : 10000
Rekening atas nama fedo berhasil dibuat dengan saldo Rp 10000.0
Rekening berhasil ditambahkan dan ditumpuk. Total rekening: 1

Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
Pilih Menu : 3
Masukkan nominal tarik : 10000
Tarik tunai Rp 10000.0 berhasil. Saldo saat ini : Rp0.0
```

Jika saldo awal kurang dari 10000 :

```
Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
Pilih Menu : 3
Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp0.0
```

B. Tantangan Multi-Akun Saat ini aplikasi Main hanya dapat menyimpan 1 akun (Rekening akunAktif = null;). Jika pengguna memilih menu "Buka Rekening Baru" untuk kedua kalinya, akun lama akan tertimpa.

- Ubah arsitektur penyimpanan di Main.java menggunakan struktur data Collection. (Gunakan ArrayList).
- Ubah logika menu "Buka Rekening": Setiap kali pengguna membuka rekening baru, objek tersebut ditambahkan ke dalam ArrayList.

- (Bonus): Tambahkan menu baru untuk "Ganti Akun", di mana sistem akan meminta nomor rekening, kemudian mencari objek yang cocok di dalam

ArrayList untuk dijadikan akunAktif

a. Class Rekening

```
1.      public boolean cocokDenganNomor(String nomor) {
2.            if (this.nomorRekening.equals(nomor)) {
3.                  return true;
4.            } else {
5.                  return false;
6.            }
7.      }
```

method cocokDenganNomor(String nomor) berfungsi sebagai alat pencocokan yang mengembalikan nilai boolean dengan membandingkan nomorRekening milik objek saat ini (this.nomorRekening) terhadap nomor yang diinputkan menggunakan .equals() — mengembalikan true jika sama persis dan false jika berbeda, yang berguna untuk mencari rekening tertentu di dalam ArrayList, misalnya pada fitur Ganti Akun.

b. Class Main

```
1.     case 5 :
2.     if (rekeningAktif.isEmpty()) {
3.     System.out.println("Belum ada rekening yang terdaftar!");
4.     } else {
5.     System.out.print("Masukkan nomor rekening yang ingin diaktifkan :
");
6.     String cariNomor = input.nextLine();
7.     boolean ditemukan = false;
8.     for (Rekening rA : rekeningAktif) {
9.     if (r.cocokDenganNomor(cariNomor)) {
10.    akunAktif = rA;
11.    ditemukan = true;
12.    System.out.println("Berhasil!
   Akun aktif sekarang atas nama " + rA.namaPemilik
13.    + " (No. Rekening: " + rA.nomorRekening + ")");
14.    break;
15.    }
16.    }
17.    if (!ditemukan) {
18.    System.out.println("Gagal: Nomor rekening " + cariNomor + " tidak
ditemukan.");
19.    }
20.    }
21.    break;
```

Case 5 (Ganti Akun) pertama-tama mengecek apakah rekeningAktif (ArrayList) masih kosong — jika ya, ditampilkan pesan bahwa belum ada rekening yang terdaftar; jika tidak kosong, user diminta memasukkan nomor rekening yang ingin diaktifkan (cariNomor), lalu disiapkan flag boolean ditemukan = false sebagai penanda status pencarian, kemudian dilakukan perulangan for-each ke setiap objek r di dalam rekeningAktif sambil mencocokkan nomornya menggunakan r.cocokDenganNomor(cariNomor) — begitu ditemukan yang cocok, objek r tersebut langsung dijadikan akunAktif yang baru, ditemukan diubah menjadi true, pesan sukses beserta nama pemilik dan nomor rekening dicetak, lalu loop dihentikan lebih awal dengan break karena nomor rekening bersifat unik sehingga tidak perlu melanjutkan pencarian; namun jika sampai akhir loop tidak ada satupun yang cocok (ditemukan tetap false), ditampilkan pesan bahwa nomor rekening yang dicari tidak ditemukan.

Output :

```
=== SISTEN PERBANKAN MINI ===

Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
Pilih Menu : 1
Masukkan No Rekening: 000
Masukkan Nama Pemilik : fedo
Masukkan Saldo Awal : 1000000
Rekening atas nama fedo berhasil dibuat dengan saldo Rp 1000000.0
Rekening berhasil ditambahkan dan ditumpuk. Total rekening: 1

Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
Pilih Menu : 1
Masukkan No Rekening: 001
Masukkan Nama Pemilik : dennis
Masukkan Saldo Awal : 200000
Rekening atas nama dennis berhasil dibuat dengan saldo Rp 200000.0
Rekening berhasil ditambahkan dan ditumpuk. Total rekening: 2
```

```
Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
Pilih Menu : 5
Masukkan nomor rekening yang ingin diaktifkan : 000
Berhasil! Akun aktif sekarang atas nama fedo (No. Rekening: 000)
```

Jika salah memasukkan nomor rekening maka :

```
Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
Pilih Menu : 5
Masukkan nomor rekening yang ingin diaktifkan : 002
Gagal: Nomor rekening 002 tidak ditemukan.
```

B. Pekan 2

a. Class Rekening :

```
1. package Pekan_2;
2.
3.     public class Rekening {
4.     String nomorRekening;
5.     String namaPemilik;
6.     double saldo;
7.
8.     public Rekening(String nomor, String nama, double saldoAwal) {
9.     nomorRekening = nomor;
10.    namaPemilik = nama;
```

```
11.     saldo = saldoAwal;
```

```
41.     public void cekInformasi() {
42.     System.out.println("--- INFO REKENING ---");
43.     System.out.println("No. Rekening : " + nomorRekening);
44.     System.out.println("Nama Pemilik : " + namaPemilik);
45.     System.out.println("Saldo Akhir  : " + saldo);
46.     System.out.println("----------------------"); 47.  }
48.
49. }
```

```
12.     System.out.println("Rekening atas nama " + namaPemilik +
" berhasil dibuat dengan saldo Rp " + saldo);
13.     }
14.
15.     public void setorTunai(double nominal) {
16.     if (nominal > 0){
17.     saldo += nominal;
18.     System.out.println("Setor tunai Rp" + nominal + " berhasil.
Saldo saat ini : Rp" + saldo);
19.     } else {
20.     System.out.println("Gagal : Nominal setor harus lebih dari
0!");
21.     }
22.     }
23.
24.     public void tarikTunai(double nominal) {
25.     if (nominal <= 10000 && nominal <= saldo) {
26.     saldo -= nominal;
27.     System.out.println("Tarik tunai Rp " + nominal + " berhasil.
Saldo saat ini : Rp" + saldo);
28.     } else {
29.     System.out.println("Transaksi Gagal: Saldo tidak mencukupi.
Saldo Anda: Rp" + saldo);
30.     }
31.     }
32.
33.     public boolean cocokDenganNomor(String nomor) {
34.     if (this.nomorRekening.equals(nomor)) {
35.     return true;
36.     } else {
37.     return false;
38.     }
39.     }
```

Kelas **Rekening** dalam package Pekan_2 berfungsi sebagai blueprint objek rekening bank, dengan tiga atribut: nomorRekening dan namaPemilik (String), serta saldo (double). Constructor Rekening(String nomor, String nama, double

saldoAwal) mengisi ketiga atribut tersebut saat objek dibuat, sekaligus mencetak pesan konfirmasi bahwa rekening berhasil dibuat.

Method setorTunai(double nominal) menambah saldo jika nominal yang disetor lebih besar dari 0, lalu mencetak saldo terbaru; jika nominal tidak valid (≤ 0), transaksi ditolak dengan pesan error.

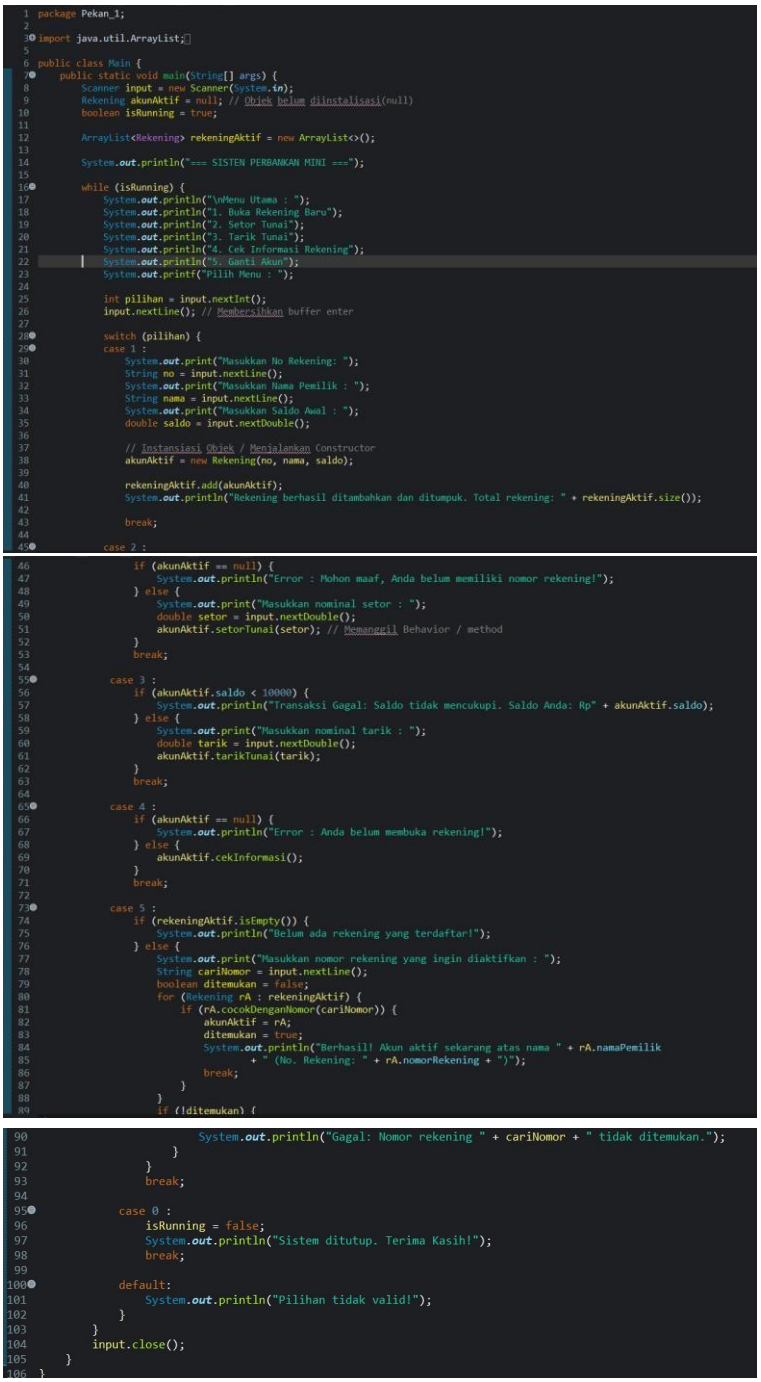
Method tarikTunai(double nominal) mengurangi saldo, tapi dengan syarat ganda: nominal harus ≤ 10000 **dan** tidak melebihi saldo yang tersedia. Kalau salah satu syarat tidak terpenuhi, transaksi gagal dan pesan menampilkan sisa saldo. (Perlu dicatat, batas 10000 di sini kemungkinan dimaksudkan sebagai limit maksimal penarikan per transaksi, bukan syarat

"saldo minimal".)

Method cocokDenganNomor(String nomor) membandingkan nomorRekening milik objek saat ini dengan nomor yang diinput, lalu mengembalikan nilai true atau false — berguna saat mencari rekening tertentu di dalam kumpulan objek (misalnya ArrayList<Rekening>).

Terakhir, cekInformasi() menampilkan seluruh data rekening (nomor, nama, saldo) dalam format yang rapi menggunakan garis pembatas.

c. Class Main



```java
package Pekan_1;

import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Rekening akunAktif = null; // Objek belum diinstalisasi(null)
        boolean isRunning = true;

        ArrayList<Rekening> rekeningAktif = new ArrayList<>();

        System.out.println("=== SISTEN PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\nMenu Utama : ");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
            System.out.printf("Pilih Menu : ");

            int pilihan = input.nextInt();
            input.nextLine(); // Membersihkan buffer enter

            switch (pilihan) {
            case 1 :
                System.out.print("Masukkan No Rekening: ");
                String no = input.nextLine();
                System.out.print("Masukkan Nama Pemilik : ");
                String nama = input.nextLine();
                System.out.print("Masukkan Saldo Awal : ");
                double saldo = input.nextDouble();

                // Instansiasi Objek / Menjalankan Constructor
                akunAktif = new Rekening(no, nama, saldo);

                rekeningAktif.add(akunAktif);
                System.out.println("Rekening berhasil ditambahkan dan ditumpuk. Total rekening: " + rekeningAktif.size());

                break;

            case 2 :
                if (akunAktif == null) {
                    System.out.println("Error : Mohon maaf, Anda belum memiliki nomor rekening!");
                } else {
                    System.out.print("Masukkan nominal setor : ");
                    double setor = input.nextDouble();
                    akunAktif.setorTunai(setor); // Memanggil Behavior / method
                }
                break;

            case 3 :
                if (akunAktif.saldo < 10000) {
                    System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + akunAktif.saldo);
                } else {
                    System.out.print("Masukkan nominal tarik : ");
                    double tarik = input.nextDouble();
                    akunAktif.tarikTunai(tarik);
                }
                break;

            case 4 :
                if (akunAktif == null) {
                    System.out.println("Error : Anda belum membuka rekening!");
                } else {
                    akunAktif.cekInformasi();
                }
                break;

            case 5 :
                if (rekeningAktif.isEmpty()) {
                    System.out.println("Belum ada rekening yang terdaftar!");
                } else {
                    System.out.print("Masukkan nomor rekening yang ingin diaktifkan : ");
                    String cariNomor = input.nextLine();
                    boolean ditemukan = false;
                    for (Rekening rA : rekeningAktif) {
                        if (rA.cocokDenganNomor(cariNomor)) {
                            akunAktif = rA;
                            ditemukan = true;
                            System.out.println("Berhasil! Akun aktif sekarang atas nama " + rA.namaPemilik
                                    + " (No. Rekening: " + rA.nomorRekening + ")");
                            break;
                        }
                    }
                    if (!ditemukan) {
                        System.out.println("Gagal: Nomor rekening " + cariNomor + " tidak ditemukan.");
                    }
                }
                break;

            case 0 :
                isRunning = false;
                System.out.println("Sistem ditutup. Terima Kasih!");
                break;

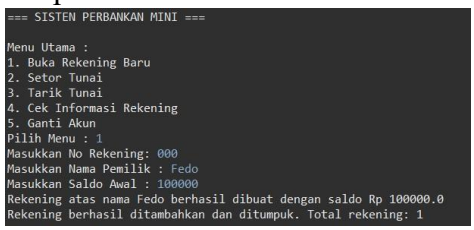
            default:
                System.out.println("Pilihan tidak valid!");
            }
        }
        input.close();
    }
}
```

Kelas **Main** dalam package Pekan_2 berfungsi sebagai program utama yang menjalankan sistem perbankan mini berbasis menu, dengan Scanner input untuk membaca input pengguna, Rekening akunAktif yang awalnya bernilai null sebagai penampung rekening yang sedang aktif, boolean isRunning sebagai flag untuk mengontrol perulangan menu, dan

ArrayList<Rekening> rekeningAktif untuk menyimpan seluruh rekening yang telah dibuat. Selama isRunning bernilai true, program menampilkan menu pilihan (Buka Rekening Baru, Setor Tunai, Tarik Tunai, Cek Informasi Rekening, Ganti Akun, dan Keluar) lalu memproses pilihan user lewat switchcase. Pada **case 1**, program meminta nomor, nama, dan saldo awal, lalu membuat objek baru dengan constructor Rekening(), menjadikannya akunAktif, dan menambahkannya ke rekeningAktif. Pada **case 2**, jika akunAktif tidak null, program meminta nominal setor lalu memanggil method setorTunai(). Pada **case 3**, program mengecek akunAktif.saldo < 10000 sebelum meminta nominal tarik dan memanggil tarikTunai() — namun kondisi ini sebenarnya keliru karena menolak transaksi ketika saldo *kurang* dari 10000, padahal validasi jumlah tarik yang benar sudah ada di dalam method tarikTunai() itu sendiri. Pada **case 4**, program memanggil cekInformasi() jika akunAktif tidak null. Pada **case 5**, jika rekeningAktif tidak kosong, program meminta nomor rekening yang ingin diaktifkan, lalu melakukan pencarian dengan for-each sambil memanggil

cocokDenganNomor() pada setiap objek; jika cocok, akunAktif diganti ke rekening tersebut. Pada **case 0**, isRunning diset false sehingga loop berhenti dan program menampilkan pesan penutup, sedangkan **default** menangani input di luar pilihan yang tersedia. Di akhir, input.close() menutup Scanner untuk melepas resource. Singkatnya, kelas ini adalah pengendali alur program (controller) yang memanfaatkan objek-objek Rekening untuk membuat, berpindah, dan bertransaksi pada rekening yang aktif, dengan satu bug logika pada validasi tarik tunai di case 3 yang perlu diperbaiki.

Output :



```
Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
Pilih Menu : 4
--- INFO REKENING ---
No. Rekening : 000
Nama Pemilik : Fedo
Saldo Akhir  : 100000.0
----------------------
```

c. Class Transaksi

```
1. package Pekan_2;
2.
3.     public class Transaksi {
4.     String idTransaksi;
5.     String jenis; 6.  double nominal; 7.
8.     // Costructor
9.     public Transaksi(String id, String jenis, double nominal) {
10.    this.idTransaksi = id;
11.    this.jenis = jenis;
12.    this.nominal = nominal;
13.    }
14.
15.    public void cetakDetail() {
16.    System.out.println("ID : " + idTransaksi + " | Jenis : " +
jenis + " | Nominal : Rp" + nominal);
17.    }
18.    }
```

Kelas **Transaksi** dalam package Pekan_2 berfungsi sebagai blueprint objek untuk merepresentasikan satu transaksi keuangan, dengan tiga atribut: idTransaksi dan jenis (String), serta nominal (double). Constructor Transaksi(String id, String jenis, double nominal) mengisi ketiga atribut tersebut menggunakan kata kunci this untuk membedakan atribut objek dengan parameter yang memiliki nama sama

(this.idTransaksi = id, this.jenis = jenis, this.nominal = nominal), tanpa mencetak pesan konfirmasi apa pun saat objek dibuat.

Method cetakDetail() menampilkan seluruh data transaksi (ID, jenis, dan nominal) dalam satu baris dengan format yang rapi menggunakan tanda pemisah "|".

Tugas/Latihan Praktikum

A. Integrasi Rekam Jejak Penarikan

- Buka kembali method tarikTunai(double nominal) di dalam file Rekening.java.
- Tambahkan logika perekaman objek Transaksi seperti pada contoh setorTunai. Pastikan jenis transaksinya dicatat sebagai "Debit" dan kodenya menggunakan prefiks "TRX-T-". Transaksi hanya boleh dicatat jika penarikan berhasil (saldo mencukupi).

a. Class Rekening

```
1.     public void tarikTunai(double nominal) {
2.     if (nominal >= 10000 && nominal <= saldo) {
3.     saldo -= nominal;
4.     String idTrx = "TRX-T-" +
   System.currentTimeMillis();
5.     Transaksi trxBaru = new Transaksi(idTrx,
   "Debit", nominal);
6.     riwayatTransaksi.add(trxBaru);
7.     System.out.println("Tarik tunai Rp " + nominal + "
berhasil. Saldo saat ini : Rp" + saldo);
8.     } else {
9.     System.out.println("Transaksi Gagal: Saldo tidak mencukupi.
Saldo Anda: Rp" + saldo);
10.    }
11.    }
```

Method tarikTunai(double nominal) berfungsi untuk mengurangi saldo rekening dengan syarat ganda yang harus dipenuhi bersamaan (&&), yaitu nominal penarikan harus kurang dari atau sama dengan 10000 dan tidak boleh melebihi saldo yang tersedia — jika kedua syarat terpenuhi, saldo dikurangi
(saldo -= nominal), lalu dibuat objek Transaksi baru dengan jenis "Debit" dan ID unik berbasis waktu (System.currentTimeMillis()), yang kemudian disimpan ke dalam riwayatTransaksi sebagai histori penarikan, dan dicetak pesan sukses beserta saldo terkini. Namun jika salah satu syarat saja tidak terpenuhi (misalnya nominal lebih dari 10000 atau melebihi saldo yang

ada), transaksi ditolak, tidak ada objek Transaksi yang dibuat, dan dicetak pesan bahwa saldo tidak mencukupi beserta saldo yang tersisa.

B. Fitur Cetak Mutasi Rekening

- Buat sebuah method baru di dalam class Rekening.java dengan nama cetakMutasi().
- Di dalam method tersebut, gunakan struktur perulangan for-each untuk menelusuri seluruh data di dalam koleksi riwayatTransaksi. Panggil method cetakDetail() milik setiap objek Transaksi. Jika riwayat kosong, cetak pesan: "Belum ada transaksi pada rekening ini"
- Buka file Main.java, tambahkan menu baru: "6. Cetak Mutasi (Riwayat)". Panggil method cetakMutasi() milik akunAktif saat pengguna memilih menu tersebut.

a. Class Rekening

```
1.     public void cetakMutasi() {
2.     if (riwayatTransaksi.isEmpty()) {
3.     System.out.println("Belum ada transaksi pada rekening
       ini");
4.     } else {
5.     for (Transaksi trx : riwayatTransaksi) {
6.     trx.cetakDetail();
7.     }
8.     }
9.     }
```

Method cetakMutasi() berfungsi untuk menampilkan seluruh riwayat transaksi yang pernah terjadi pada suatu rekening. Pertama, method ini mengecek apakah riwayatTransaksi (list ArrayList<Transaksi>) kosong dengan isEmpty() — jika benar kosong, dicetak pesan bahwa belum ada transaksi pada rekening tersebut. Namun jika tidak kosong, program

menggunakan perulangan for-each untuk mengiterasi setiap objek Transaksi trx di dalam riwayatTransaksi, dan pada setiap iterasi memanggil method trx.cetakDetail() milik objek tersebut, yang akan menampilkan ID, jenis, dan nominal dari masing-masing transaksi secara berurutan.

b. Class Main

```
1. case 6 :
2.                         akunAktif.cetakMutasi();
3.                         break;
```

Blok case 6 ini menambahkan opsi menu baru pada switch-case di kelas Main, yang berfungsi untuk mencetak mutasi (riwayat transaksi) dari rekening yang sedang aktif. Ketika user memilih menu 6, program langsung memanggil
akunAktif.cetakMutasi(), yang akan menampilkan seluruh histori transaksi (setor dan tarik) milik akunAktif — atau pesan "belum ada transaksi" jika riwayatnya masih kosong. Setelah itu, break menghentikan eksekusi switch agar tidak lanjut memeriksa case-case berikutnya.

Output :

```
Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi
Pilih Menu : 6
ID : TRX-S-1789664955328 | Jenis : Kredit | Nominal : Rp100000.0
ID : TRX-T-1789664961114 | Jenis : Debit | Nominal : Rp100000.0
```

C. Pekan 3

1. Class Rekening :

```java
package Pekan_3;

import java.util.ArrayList;

public class Rekening {
    // 1, mengunci atribut dengan 'Private'
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin; // data sensitif

    // Implementasi Asosiasi (1-to-many)
    ArrayList<Transaksi>riwayatTransaksi;

    // 2. Modifikasi Constructor untuk menerima PIN awal
    public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
            this.nomorRekening = nomor;
            this.namaPemilik = nama;
            this.setSaldo(saldoAwal);

            // Validasi PIN di dalam constructor
            if (pinAwal.length() == 6) {
                  this.pin = pinAwal;
            } else {
                  System.out.println("Peringatan : PIN harus 6 digit! Menggunakan PIN default 123456");
                  this.pin = "123456";
            }
    }

            // 3. Getter untuk atribut yang diizinkan dibaca publik
            public String getNomorRekening() {
                  return nomorRekening;
            }

            public String getNamaPemilik() {
                  return namaPemilik;
            }

            // 4. Method Otentikasi Internal (Validasi Enkapsulasi()
            public boolean otentikasi(String inputPin) {
                  return this.pin.equals(inputPin);
            }
```

Kode ini merupakan class **Rekening** dalam package **Pekan_3**, yang merupakan versi pengembangan dari kode sebelumnya dengan menerapkan konsep **enkapsulasi**. Perbedaan utamanya terletak pada atribut nomorRekening, namaPemilik, saldo, dan pin yang kini dideklarasikan sebagai **private**, sehingga tidak dapat diakses langsung dari luar class dan hanya bisa dimanipulasi melalui method tertentu, guna melindungi keamanan dan integritas data, terutama data sensitif seperti pin. Atribut **riwayatTransaksi** tetap dipertahankan sebagai implementasi asosiasi satu-ke-banyak antara rekening dan transaksi.

Pada **constructor Rekening**, parameter yang diterima kini bertambah dengan **pinAwal**, sehingga saat objek rekening dibuat, PIN awal juga harus disertakan. Nilai saldoAwal tidak langsung diberikan ke atribut saldo, melainkan melalui pemanggilan method setSaldo(saldoAwal), yang mengindikasikan adanya validasi tambahan pada proses pengisian saldo (meski implementasi method tersebut tidak ditampilkan). Selain itu, constructor juga melakukan validasi terhadap panjang PIN yang dimasukkan; jika PIN berjumlah 6 digit maka akan disimpan sebagai pin rekening, namun jika tidak sesuai, sistem akan menampilkan peringatan dan secara otomatis menggunakan PIN default "123456" sebagai bentuk penanganan kesalahan input.

Selanjutnya terdapat dua method **getter**, yaitu getNomorRekening() dan getNamaPemilik(), yang berfungsi untuk mengizinkan pembacaan nilai atribut privat nomorRekening dan namaPemilik dari luar class tanpa harus membuka akses langsung ke atributnya, sesuai prinsip enkapsulasi bahwa hanya atribut tertentu yang boleh diakses publik.

Terakhir, terdapat method **otentikasi(String inputPin)** yang berfungsi sebagai validasi keamanan internal, dengan cara membandingkan nilai inputPin yang dimasukkan pengguna dengan nilai pin yang tersimpan di dalam objek menggunakan .equals(). Method ini mengembalikan nilai

boolean true jika PIN cocok dan false jika tidak, sehingga dapat digunakan sebagai syarat sebelum mengizinkan proses transaksi seperti penarikan tunai, tanpa pernah mengekspos nilai pin secara langsung ke luar class.

Mengapa kita tidak membuat method setSaldo() dan getPin() ?

Karna, Saldo adalah atribut yang nilainya harus berubah melalui **aturan bisnis tertentu**, bukan diubah sembarangan. Jika kita membuat method setSaldo() yang bisa dipanggil bebas dari luar class, maka siapa pun bisa mengubah saldo. Itu sebabnya perubahan saldo hanya boleh terjadi melalui method seperti setorTunai() dan tarikTunai(), yang di dalamnya sudah ada validasi (nominal harus positif, saldo mencukupi, dan sebagainya). Kalaupun ada method setSaldo() seperti yang dipanggil di constructor, sifatnya biasanya private. Sedangkan pin, Karena **pin adalah data sensitif**. Jika dibuatkan getter publik, maka nilai pin bisa "dibocorkan" dan dibaca oleh class atau pihak lain di luar Rekening, yang jelas berbahaya untuk keamanan data nasabah. Prinsip enkapsulasi di sini bukan cuma "menyembunyikan atribut", tapi juga **membatasi cara mengaksesnya**. Karena itu, alih-alih menyediakan getPin() untuk membaca pin secara langsung, dibuatlah method otentikasi(String inputPin) yang hanya mengembalikan hasil **perbandingan** (true/false), tanpa pernah mengekspos nilai pin yang sebenarnya. Dengan begitu, sistem tetap bisa memverifikasi PIN pengguna tanpa risiko nilai pin bisa "dicuri" atau dibaca langsung oleh kode dari luar.

2. Class Main

Untuk kelas Main tidak ada perubahan dari modul, tetapi terdapat perubahan pada bagian tugas/latihan praktikum. Jadi kelas main masih sama seperti pada pekan sebelumnya

3. Class Transaksi

```java
package Pekan_3;

public class Transaksi {
    // 1. Mengubah semua atribut menjadi private
    private String idTransaksi;
    private String jenis;
    private double nominal;

    // Costructor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    // 2. Hanya menyediakan getter (Read-only)
    public String getIdTransaksi() {
        return idTransaksi;
    }

    public String getJenis() {
        return jenis;
    }

    public double getNominal() {
        return nominal;
    }

    public void cetakDetail() {
        System.out.println("ID : " + idTransaksi + " | Jenis : " + jenis + " | Nominal : Rp" + nominal);
    }
}
```

Kode ini merupakan class Transaksi dalam package Pekan_3, yang merupakan versi enkapsulasi dari class Transaksi sebelumnya. Ketiga atribut, yaitu idTransaksi, jenis, dan nominal, kini dijadikan private, sehingga tidak dapat diakses atau diubah langsung dari luar class, dengan tujuan menjaga agar data transaksi tetap konsisten dan tidak bisa dimanipulasi sembarangan setelah objek dibuat. Constructor tetap berfungsi sama seperti sebelumnya, yaitu menerima parameter id, jenis, dan nominal untuk menginisialisasi ketiga atribut tersebut saat objek Transaksi dibuat.

Untuk mengakses nilai-nilai atribut privat tersebut, disediakan tiga method getter, yaitu getIdTransaksi(), getJenis(), dan getNominal(), yang sifatnya read-only atau hanya untuk membaca nilai, tanpa disertai method

setter. Hal ini sengaja dilakukan karena data transaksi seharusnya bersifat immutable (tidak dapat diubah) setelah tercatat, sebab representasi riwayat transaksi seharusnya tetap dan tidak boleh diedit demi menjaga keakuratan catatan keuangan. Terakhir, method cetakDetail() tetap dipertahankan untuk menampilkan informasi transaksi secara lengkap, meliputi ID, jenis, dan nominal transaksi.

Tugas/Latihan Praktikum

a. Menu 1 (Buka Rekening): Sebelum menginstansiasi objek baru, minta user menginputkan PIN (String angka 6 digit).

b. Keamanan Transaksi: Perbarui Menu 3 (Tarik Tunai) dan Menu 6 (Cetak Mutasi). Sebelum sistem mengeksekusi kedua method tersebut, sistem harus meminta user memasukkan PIN.

• Panggil fungsi akunAktif.otentikasi(pinYangDiinput).

• Jika mengembalikan nilai true, jalankan transaksi.

• Jika false, batalkan dan cetak: "Akses Ditolak: PIN yang Anda masukkan salah!".

Jawaban :

a. Kode program class Main :

```
case 1 :
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik : ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal : ");
    double saldo = input.nextDouble();
    input.nextLine();
    System.out.println("Masukkan PIN Awal : ");
    String pin = input.nextLine();
    // Instansiasi Objek / Menjalankan Constructor
    akunAktif = new Rekening(no, nama, saldo, pin);

    rekeningAktif.add(akunAktif);
```

```
15.     System.out.println("Rekening berhasil ditambahkan dan
        ditumpuk. Total rekening: " + rekeningAktif.size());
16.
17.     Break;
```

Kode ini merupakan potongan **case 1** dari program Pekan_3, yang berfungsi untuk menambahkan rekening baru. Program meminta input dari pengguna berupa nomor rekening, nama pemilik, dan saldo awal menggunakan input.nextLine() dan input.nextDouble(). Setelah input saldo (yang bertipe double) dibaca, ditambahkan input.nextLine() tambahan yang berfungsi untuk **membersihkan sisa karakter newline** di buffer input, agar input String berikutnya (pin) tidak terlewat atau terbaca kosong akibat sisa enter dari nextDouble().

Perbedaan utama dari versi Pekan_2 adalah adanya input tambahan berupa **pin**, yang diminta melalui input.nextLine(), sesuai dengan constructor Rekening pada Pekan_3 yang kini mewajibkan parameter pin saat objek dibuat. Selanjutnya, objek akunAktif diinstansiasi dengan memanggil constructor new Rekening(no, nama, saldo, pin), lalu dimasukkan ke dalam list rekeningAktif menggunakan .add(). Terakhir, program menampilkan pesan konfirmasi bahwa rekening berhasil ditambahkan beserta jumlah total rekening yang tersimpan, sebelum keluar dari case menggunakan break.

b. Kode program class Main :

```
1.  case 3 :
2.      System.out.println("Masukkan PIN : ");
3.      String pinYangDiinput = input.nextLine();
4.      if (akunAktif.otentikasi(pinYangDiinput) == false) {
5.              System.out.println("Akses Ditolak: PIN yang Anda
                masukkan salah!");
6.      } else {
7.              if (akunAktif.getSaldo() < 10000) {
8.              System.out.println("Transaksi Gagal: Saldo tidak
                mencukupi. Saldo Anda: Rp" + akunAktif.getSaldo());
9.      } else {
10.             System.out.print("Masukkan nominal tarik : ");
11.             double tarik = input.nextDouble();
12.             akunAktif.tarikTunai(tarik);
13.     }
14.     }
```

```
15.    break;
16.
17. case 6 :
18.    System.out.print("Masukkan PIN : ");
19.    String pinYangDiinput1 = input.nextLine();
20.    if (akunAktif.otentikasi(pinYangDiinput1) == false) {
21.           System.out.println("Akses Ditolak: PIN yang Anda
               masukkan salah!");
22.    } else {
23.           akunAktif.cetakMutasi();
24.    }
25.    break;
26.
```

Kode ini merupakan potongan **case 3** (tarik tunai) dan **case 6** (cetak mutasi) dari program Pekan_3, yang kini menambahkan lapisan **keamanan berupa verifikasi PIN** sebelum transaksi sensitif dapat dilakukan. Pada kedua case, program terlebih dahulu meminta pengguna memasukkan PIN melalui input.nextLine(), kemudian nilai tersebut dicek menggunakan method otentikasi() milik objek akunAktif. Jika hasil otentikasi bernilai false (PIN salah), maka sistem akan menampilkan pesan "Akses Ditolak" dan proses transaksi tidak dilanjutkan.

Pada **case 3**, jika PIN terbukti benar, program akan melanjutkan ke pengecekan saldo menggunakan getSaldo() (bukan mengakses atribut saldo secara langsung, karena kini bersifat private). Jika saldo kurang dari 10.000, transaksi dinyatakan gagal, namun jika mencukupi, pengguna diminta memasukkan nominal penarikan yang kemudian diproses melalui method tarikTunai(). Pada **case 6**, jika PIN valid, program akan langsung memanggil method cetakMutasi() untuk menampilkan riwayat transaksi rekening tersebut.

Perbedaan utama dari versi Pekan_2 terletak pada penerapan konsep **enkapsulasi dan keamanan data**, di mana akses ke atribut saldo kini melalui getter (getSaldo()), serta transaksi penting seperti tarik tunai dan pengecekan mutasi tidak lagi bisa dilakukan secara bebas, melainkan harus melalui proses verifikasi PIN terlebih dahulu.

OUTPUT :

a. Output soal pertama

Case 1 :

```
=== SISTEN PERBANKAN MINI ===

Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi
Pilih Menu : 1
Masukkan No Rekening: 000
Masukkan Nama Pemilik : Fedo
Masukkan Saldo Awal : 200000
Masukkan PIN : 060407
Rekening berhasil ditambahkan dan ditumpuk. Total rekening: 1
```

b. Output soal kedua

Case 3 :

```
Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi
Pilih Menu : 3
Masukkan PIN : 060407
Masukkan nominal tarik : 50000
Tarik tunai Rp 50000.0 berhasil. Saldo saat ini : Rp150000.0
```

Case 6 :

```
Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi
Pilih Menu : 6
Masukkan PIN : 060407
ID : TRX-T-1790408186320 | Jenis : Debit | Nominal : Rp50000.0
```

D. Pekan 4

1. Class Rekening :

```
1. package Pekan_4;
2.
3. import java.util.ArrayList;
4.
5.     public class Rekening {
6.     // Bagian atas dari class Rekening.java
7.     private String nomorRekening;
8.     private String namaPemilik;
9.     private String pin; // data sensitif 10.
11.    // Gunakan prptected agar Subclass bisa mengaksesnya langsung
12.    protected double saldo;
13.    protected ArrayList<Transaksi>riwayatTransaksi; 14.
15.  // Isi konstruktor dan methode lainnya TETAP SAMA seperti Modul
   3
```

Kode ini merupakan bagian awal dari class **Rekening** dalam package **Pekan_4**, yang merupakan pengembangan lebih lanjut dari konsep enkapsulasi pada Pekan_3, dengan tambahan persiapan untuk konsep **pewarisan (inheritance)**. Atribut nomorRekening, namaPemilik, dan pin tetap dijadikan **private**, karena ketiganya bersifat data pribadi dan sensitif yang tidak perlu maupun tidak boleh diakses langsung oleh class lain, termasuk subclass sekalipun.

Namun, atribut **saldo** dan **riwayatTransaksi** kini diubah modifier aksesnya menjadi **protected**, bukan lagi private seperti pada Pekan_3. Perubahan ini bertujuan agar kedua atribut tersebut dapat **diakses langsung oleh subclass** (class turunan) dari Rekening, tanpa harus melalui getter/setter, karena nantinya subclass mungkin memerlukan akses langsung terhadap saldo maupun riwayat transaksi untuk keperluan logika bisnis tambahan yang spesifik pada turunannya. Sementara itu, bagian constructor dan method lainnya dinyatakan tetap sama seperti pada Modul 3 (Pekan_3), sehingga logika validasi PIN, pembuatan objek, serta method-method seperti setorTunai(), tarikTunai(), otentikasi(), dan lainnya tidak mengalami perubahan.

1. Class Main

Untuk kelas Main tidak ada perubahan dari modul, tetapi terdapat perubahan pada bagian tugas/latihan praktikum. Jadi kelas main masih sama seperti pada pekan sebelumnya.

2. Class Transaksi

Untuk kelas Transaksi tidak ada perubahan dari modul, Jadi kelas main masih sama seperti pada pekan sebelumnya.

3. Class RekeningTabungan

```
1. package Pekan_4;
2.
3. public class RekeningTabungan extends Rekening {
4.
      5.     // Atribut spesfik yang hanya dimiliki oleh Tabungan
      6.     private double sukuBunga;
7.
8.
            9.     // Contructor Subclass
            10.    public RekeningTabungan (String nomor, String
            nama, double saldoAwal, String pinAwal, double
            sukuBunga) {
            11.    // super() memanggl constructor kelas induk
            (Rekening), WAJIB berada di baris pertama
            12.    super(nomor, nama, saldoAwal, pinAwal);
            13.    this.sukuBunga = sukuBunga; 14.  }
15.
            16.    public void tambahBungaAkhirBulan() {
            17.    // Menghitung Bunga
            18.    // Mengapa bisa mengakses saldo secara langsung
            dari class RekeningTabungan ?
19.         double nominalBunga = saldo * (sukuBunga / 100);
20.         saldo += nominalBunga;
21.
22.         // Mencatat riwayat transaksi
23.         String idTrx = "TRX-B" + System.currentTimeMillis();
24.         riwayatTransaksi.add(new Transaksi(idTrx, "Bunga",
            nominalBunga));
25.
26.    System.out.println("Bunga " + sukuBunga + "% berhasil
   ditambahkan : Rp" + nominalBunga);
27.
28.  }
29.
30. }
```

Kode ini merupakan class RekeningTabungan dalam package Pekan_4, yang merupakan subclass (class turunan) dari class Rekening menggunakan keyword extends, sehingga menerapkan konsep pewarisan (inheritance) dalam OOP. Class ini memiliki atribut tambahan yang spesifik hanya dimiliki oleh rekening tabungan, yaitu sukuBunga bertipe double dan bersifat private, karena hanya relevan dan digunakan di dalam class ini sendiri.

Pada constructor RekeningTabungan, parameter yang diterima meliputi nomor, nama, saldoAwal, pinAwal, dan sukuBunga. Di baris pertama constructor, dipanggil super(nomor, nama, saldoAwal, pinAwal), yang berfungsi untuk memanggil constructor milik class induk (Rekening) guna menginisialisasi atribut-atribut yang diwariskan dari Rekening. Pemanggilan super() ini wajib diletakkan di baris pertama constructor subclass, karena proses inisialisasi objek induk harus selesai terlebih dahulu sebelum subclass menambahkan inisialisasi miliknya sendiri, dalam hal ini adalah atribut sukuBunga.

Selanjutnya terdapat method tambahBungaAkhirBulan(), yang berfungsi untuk menghitung dan menambahkan bunga ke saldo rekening pada akhir bulan. Nominal bunga dihitung dari saldo * (sukuBunga / 100), kemudian hasilnya ditambahkan langsung ke saldo. Perlu diperhatikan bahwa atribut saldo dapat diakses secara langsung di class ini (tanpa getter/setter) karena pada class Rekening, saldo dideklarasikan dengan modifier protected, bukan private, sehingga dapat diwariskan dan diakses langsung oleh subclassnya, termasuk RekeningTabungan ini. Hal yang sama berlaku untuk atribut riwayatTransaksi, yang juga bersifat protected, sehingga method ini bisa langsung memanggil riwayatTransaksi.add() untuk mencatat transaksi baru bertipe "Bunga" dengan id transaksi unik berdasarkan waktu sistem. Terakhir, program menampilkan pesan konfirmasi bahwa bunga berhasil ditambahkan beserta nominalnya.

5. Class RekeningGiro

```
1. package Pekan_4;
2.
       3.     public class RekeningGiro extends Rekening{
       4.     private double batasOverdraft;
5.
              6.     public RekeningGiro (String nomor, String
              nama, double saldoAwal, String pinAwal, double
              batasOverdraft) {
              7.     // Memanggil inisialisasi dasar dari
              Superclass
              8.     super(nomor, nama, saldoAwal, pinAwal);
              9.     this.batasOverdraft = batasOverdraft; 10.
              }
11.
12.    // Getter khusus Giro
13.    public double getBatasOverdraft() {
14.    return batasOverdraft; 15.  }
16.
17.  // Catatan Penarikan hingga limit overdraft akan kita
   selesaikan di modul 5
18.
19. }
```

Kode ini merupakan class RekeningGiro dalam package Pekan_4, yang juga merupakan subclass dari class Rekening menggunakan extends, sama seperti RekeningTabungan sebelumnya, sehingga menerapkan konsep pewarisan (inheritance). Class ini memiliki atribut tambahan yang spesifik untuk rekening giro, yaitu batasOverdraft bertipe double dan bersifat private, yang berfungsi untuk menyimpan batas maksimal penarikan yang diperbolehkan meski saldo mencapai minus (overdraft).

Pada constructor RekeningGiro, parameter yang diterima meliputi nomor, nama, saldoAwal, pinAwal, dan batasOverdraft. Sama seperti pada RekeningTabungan, baris pertama constructor memanggil super(nomor, nama, saldoAwal, pinAwal) untuk menjalankan inisialisasi dasar dari superclass (Rekening), sebelum kemudian menginisialisasi atribut batasOverdraft yang khusus dimiliki oleh RekeningGiro.

Selanjutnya disediakan method getBatasOverdraft() sebagai getter khusus untuk membaca nilai batasOverdraft dari luar class, mengikuti prinsip enkapsulasi yang tetap diterapkan meskipun class ini adalah subclass.

Terakhir, terdapat komentar yang menandakan bahwa logika penarikan dana hingga melewati saldo (memanfaatkan batas overdraft) belum diimplementasikan pada tahap ini dan direncanakan akan diselesaikan pada modul berikutnya (Modul 5), sehingga class ini untuk sementara hanya menyimpan data batas overdraft tanpa mengubah perilaku transaksi yang diwariskan dari Rekening.

Tugas/Latihan Praktikum

1. Pembuatan Multi-Produk

Modifikasi case 1: Buka Rekening Baru di Main.java.

a. Setelah user menginputkan Saldo Awal dan PIN, berikan opsi produk: "Pilih

Produk: 1. Tabungan Umum | 2. Giro Bisnis"

b. Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek RekeningTabungan.

c. Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek RekeningGiro.

d. Simpan objek tersebut ke dalam daftarRekening (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

2. Simulasi Waktu Akhir Bulan

a. Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.

b. Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof.

c. Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().

d. Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.

Jawaban :

1. Kode program class Main :

```
switch (pilihan) {
case 1 :
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik : ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal : ");
    double saldo = input.nextDouble();
    input.nextLine();
    System.out.print("Masukkan PIN : ");
    String pin = input.nextLine();
    System.out.print("Pilih Produk : 1. Tabungan Umum | 2. Giro Bisnis :");
    int pil = input.nextInt();
    if (pil == 1) {
        System.out.print("Masukkan Suku Bunga (%): ");
        double sukuBunga = input.nextDouble();
        akunAktif = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);
    } else if (pil == 2) {
        System.out.print("Masukkan Batas Overdraf: ");
        double batasOverdraf = input.nextDouble();
        akunAktif = new RekeningGiro(no, nama, saldo, pin, batasOverdraf);
    } else {
        System.out.println("Pilihan tidak valid");
    }

    if (akunAktif != null) {
        rekeningAktif.add(akunAktif);
        System.out.println("Rekening berhasil ditambahkan. Total rekening: " + rekeningAktif.size());
    }
    break;
```

Bagian awal (input nomor rekening, nama, saldo awal, dan PIN) tidak mengalami perubahan dan tetap sama seperti Pekan_3. Penambahan baru dimulai setelah input PIN, yaitu program kini menampilkan pilihan jenis produk rekening kepada pengguna melalui System.out.print("Pilih Produk : 1. Tabungan Umum | 2. Giro Bisnis :"), yang dibaca menggunakan input.nextInt() dan disimpan ke variabel pil.

Berdasarkan pilihan tersebut, ditambahkan struktur if-else untuk menentukan jenis objek yang akan dibuat, memanfaatkan konsep pewarisan (inheritance) dari Pekan_4: jika pil == 1, program meminta input tambahan berupa suku bunga, lalu membuat objek dari subclass RekeningTabungan; jika pil == 2, program meminta input batas overdraft, lalu membuat objek dari subclass RekeningGiro; dan jika input tidak sesuai (bukan 1 atau 2), program menampilkan pesan "Pilihan tidak valid" tanpa membuat objek apa pun, sehingga akunAktif tetap bernilai null.

Penambahan penting lainnya adalah adanya pengecekan if (akunAktif != null) sebelum proses rekeningAktif.add(akunAktif) dan penampilan pesan sukses dijalankan. Penambahan ini diperlukan sebagai antisipasi terhadap kemungkinan pilihan produk yang tidak valid, sehingga program tidak keliru menambahkan data kosong (null) ke dalam list rekeningAktif atau

menampilkan pesan sukses yang salah, sesuatu yang tidak perlu ada pada Pekan_3 karena di sana hanya ada satu jenis objek Rekening yang pasti langsung dibuat tanpa percabangan pilihan produk.

Output :

```
=== SISTEN PERBANKAN MINI ===

Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi
7. Simulasi Akhir Bulan (Khusus Tabungan)
Pilih Menu : 1
Masukkan No Rekening: 0000
Masukkan Nama Pemilik : Fedo
Masukkan Saldo Awal : 200000
Masukkan PIN : 000000
Pilih Produk : 1. Tabungan Umum | 2. Giro Bisnis :1
Masukkan Suku Bunga (%): 20
Rekening berhasil ditambahkan. Total rekening: 1
```

Jika salah menginput pilihan :

```
=== SISTEN PERBANKAN MINI ===

Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi
7. Simulasi Akhir Bulan (Khusus Tabungan)
Pilih Menu : 1
Masukkan No Rekening: 000
Masukkan Nama Pemilik : fedo
Masukkan Saldo Awal : 200000
Masukkan PIN : 000000
Pilih Produk : 1. Tabungan Umum | 2. Giro Bisnis :3
Pilihan tidak valid
```

2. Kode program class Main :

```
case 7 :
    for (Rekening tabungan : rekeningAktif) {
        System.out.println("No: " + tabungan.getNomorRekening() + ", Nama: " + tabungan.getNamaPemilik());

        if (tabungan instanceof RekeningTabungan) {
            RekeningTabungan rekeningTabungan = (RekeningTabungan) tabungan;   // downcasting
            rekeningTabungan.tambahBungaAkhirBulan();
        } else {
            System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan");
        }
    }
break;
```

Kode ini merupakan potongan **case 7**, yang berfungsi untuk menjalankan fitur penambahan bunga akhir bulan pada seluruh rekening yang tersimpan, dengan memanfaatkan konsep **polimorfisme** dan **downcasting**. Program menggunakan perulangan **for-each** for (Rekening tabungan : rekeningAktif) untuk mengakses setiap objek rekening yang ada di dalam list rekeningAktif satu per satu, meskipun list tersebut sebenarnya bisa berisi campuran objek RekeningTabungan maupun RekeningGiro, karena keduanya merupakan turunan dari class Rekening. Pada setiap iterasi, program menampilkan informasi nomor rekening dan nama pemilik menggunakan method getNomorRekening() dan getNamaPemilik() yang diwarisi dari class Rekening.

Selanjutnya, dilakukan pengecekan menggunakan instanceof melalui kondisi if (tabungan instanceof RekeningTabungan), yang berfungsi untuk memeriksa apakah objek rekening yang sedang diproses benar-benar merupakan objek dari tipe **RekeningTabungan**, mengingat method tambahBungaAkhirBulan() hanya tersedia pada subclass tersebut dan tidak dimiliki oleh Rekening maupun RekeningGiro. Jika hasil pengecekan bernilai true, maka dilakukan **downcasting**, yaitu mengubah tipe objek tabungan (bertipe Rekening) menjadi tipe **RekeningTabungan** melalui (RekeningTabungan) tabungan, sehingga objek tersebut dapat memanggil method tambahBungaAkhirBulan() yang spesifik dimiliki subclass tersebut. Downcasting ini diperlukan karena secara default variabel tabungan hanya
"dikenali" sebagai objek Rekening oleh compiler, sehingga tidak bisa langsung mengakses method yang hanya ada di subclass tanpa proses casting terlebih dahulu.

Namun, jika objek yang diperiksa bukan merupakan RekeningTabungan (misalnya RekeningGiro), maka program akan menampilkan pesan bahwa fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan, sehingga mencegah terjadinya error ClassCastException yang bisa muncul apabila downcasting dipaksakan pada objek yang tipenya tidak sesuai.

Output :

Menggunakan lanjutan dari case 1 diatas :

```
Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi
7. Simulasi Akhir Bulan (Khusus Tabungan)
Pilih Menu : 7
No: 0000, Nama: Fedo
Bunga 20.0% berhasil ditambahkan : Rp40000.0
```

Jika rekening yang dipilih adalah rekening giro :

```
Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi
7. Simulasi Akhir Bulan (Khusus Tabungan)
Pilih Menu : 7
No: 0000, Nama: Fedo
Bunga 20.0% berhasil ditambahkan : Rp48000.0
No: 0000, Nama: Fedo
Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan
```

E. Pekan 5

1. Class Rekening :

```java
// Mengubah void menjadi boolean
    public boolean tarikTunai(double nominal) {
          if (nominal <= 0) {
                System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
                return false;
          } else if (nominal > saldo){
                System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
                return false;
          } else {
                saldo -= nominal;

                String idTrx = "TRX-T-" + System.currentTimeMillis();
                riwayatTransaksi.add(new Transaksi(idTrx, "Debit", nominal));

                System.out.println("Tarik tunai Rp" + nominal + " berhasil. Sisa saldo: Rp" + saldo);
```

```
16.                 return true; // Transaksi sukses
17.             }
18.     }
```

Kode Pekan_5 ini adalah method tarikTunai() pada class Rekening yang tipe kembaliannya diubah dari void (Pekan_4) menjadi boolean, sehingga method melaporkan hasil transaksi ke pemanggilnya: true jika berhasil dan false jika gagal. Dengan begitu, Main atau subclass dapat mengambil keputusan berdasarkan hasil penarikan, tidak hanya mengandalkan pesan di layar.

Validasi diubah dari satu kondisi nominal >= 10000 && nominal <= getSaldo() dengan satu pesan kegagalan, menjadi struktur if-else if-else yang lebih spesifik. Kondisi pertama menolak nominal kurang dari atau sama dengan 0, dan kondisi kedua menolak nominal yang melebihi saldo, masing-masing dengan pesan sesuai penyebabnya lalu mengembalikan false. Batas minimal Rp10.000 juga dihapus agar method lebih umum, sehingga aturan khusus seperti limit overdraft pada RekeningGiro dapat ditentukan oleh subclass melalui override.

Jika penarikan valid, saldo dikurangi langsung dengan saldo -= nominal, bukan lagi lewat setSaldo(getSaldo() - nominal), karena atribut saldo bersifat protected sehingga lebih ringkas. Transaksi tetap dicatat dengan membuat id unik berawalan “TRX-T-” dari System.currentTimeMillis(), lalu objek Transaksi berjenis “Debit” langsung ditambahkan ke riwayatTransaksi tanpa variabel perantara. Terakhir, program menampilkan pesan sukses dengan redaksi “Sisa saldo” (sebelumnya “Saldo saat ini”) dan mengembalikan true.

2. Class Main

Untuk kelas Main tidak ada perubahan dari modul, tetapi terdapat perubahan pada bagian tugas/latihan praktikum. Jadi kelas main masih sama seperti pada pekan sebelumnya.

3. Class Transaksi

Untuk kelas Transaksi tidak ada perubahan dari modul, Jadi kelas Transaksi masih sama seperti pada pekan sebelumnya.

4. Class RekeningTabungan

Untuk kelas RekeningTabungan tidak ada perubahan dari modul, Jadi kelas RekeningTabungan masih sama seperti pada pekan sebelumnya

5. Class RekeningGiro

```
1.  package Pekan_5;
2.
3.  public class RekeningGiro extends Rekening{
4.      private double batasOverdraft;
5.
6.      public RekeningGiro (String nomor, String nama, double
    saldoAwal, String pinAwal, double batasOverdraft) {
7.              // Memanggil inisialisasi dasar dari Superclass
8.              super(nomor, nama, saldoAwal, pinAwal);
9.              this.batasOverdraft = batasOverdraft;
10.     }
11.
12.     // IMPLEMENTASI POLIMORFISME (OVERRIDING)
13.     @Override
14.     public boolean tarikTunai(double nominal) {
15.             if (nominal <= 0) {
16.                     System.out.println("Gagal: Nominal tarik Giro
    harus lebih dari 0!");
17.                     return false;
18.             }
19.             // Logika Khusus Giro: Boleh ditarik asalkan nominal
    tidak melebihi (saldo + Overdraft)
20.             else if (nominal > (saldo + batasOverdraft)) {
21.                     System.out.println("Transaksi Gagal: Melebihi
    batas overdraft! Maksimal penarikan: Rp" + (saldo +
    batasOverdraft));
22.                     return false;
23.             } else {
24.                     saldo -= nominal; // Saldo bisa bernilai negatif
    disini (memakan overdraft)
25.
26.                     String idTrx = "TRX-G-" +
    System.currentTimeMillis();
27.                     riwayatTransaksi.add(new Transaksi(idTrx, "Debit
    Giro", nominal));
28.
29.                     System.out.println("Tarik tunai Giro Rp" +
    nominal + " berhasil. Saldo saat ini: Rp" + saldo);
30.                     return true;
31.             }
32.     }
```

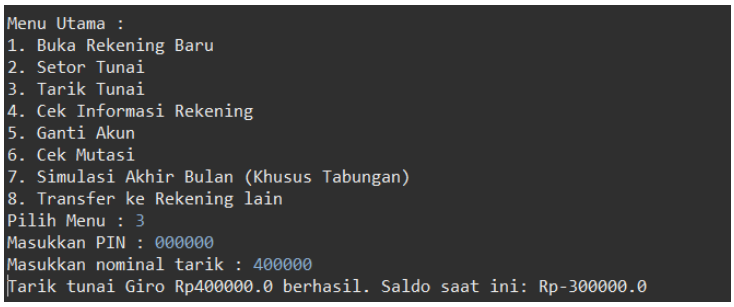
```
33.
34.     public double getBatasOverdraft() {
35.           return batasOverdraft;
36.     }
37.     // Catatan Penarikan hingga limit overdraft akan kita
    selesaikan di modul 5
38.
39. }
```

Kode Pekan_5 ini adalah class RekeningGiro, subclass dari Rekening (extends), yang dibanding Pekan_4 mendapat tambahan method tarikTunai() dengan anotasi @Override. Pada Pekan_4, class ini hanya menyimpan atribut private batasOverdraft (batas maksimal saldo boleh minus) beserta getter-nya, sehingga penarikan masih memakai aturan bawaan Rekening yang tidak mengizinkan saldo negatif. Constructor-nya memanggil super(nomor, nama, saldoAwal, pinAwal) untuk menginisialisasi data dasar dari Rekening, lalu mengisi batasOverdraft yang khusus dimiliki Giro.

Method tarikTunai() menerapkan polimorfisme (overriding): nama dan tipe kembaliannya (boolean) sama dengan milik Rekening, tetapi perilakunya disesuaikan dengan rekening giro. Nominal kurang dari atau sama dengan 0 ditolak dengan pesan khusus Giro dan mengembalikan false. Perbedaan terpentingnya ada pada kondisi nominal > (saldo + batasOverdraft), yaitu penarikan hanya ditolak jika melebihi saldo ditambah batas overdraft (bukan hanya saldo), dengan pesan yang menampilkan jumlah maksimal yang boleh ditarik, lalu mengembalikan false. Jika valid, saldo dikurangi langsung dengan saldo -= nominal (bisa karena saldo bersifat protected) dan boleh bernilai negatif karena memakai fasilitas overdraft.

Transaksi dicatat dengan id berawalan “TRX-G-” dan jenis “Debit Giro”, berbeda dari rekening biasa (“TRX-T-” dan “Debit”), sehingga mutasi dapat membedakannya. Setelah itu ditampilkan pesan sukses beserta saldo saat ini dan method mengembalikan true. Terakhir, getter getBatasOverdraft() tetap dipertahankan, sedangkan komentar “akan diselesaikan di modul 5” sudah tidak relevan dan sebaiknya dihapus karena fiturnya kini sudah diimplementasikan.

6. Uji coba



Gambar ini menunjukkan hasil eksekusi program Pekan_5 saat pengguna memilih menu 3 (Tarik Tunai) pada akun bertipe RekeningGiro. Setelah PIN (000000) diterima oleh method otentikasi(), pengguna menarik Rp400.000 dan program menampilkan pesan "Tarik tunai Giro Rp400000.0 berhasil". Kata "Giro" pada pesan menandakan bahwa yang dijalankan adalah method tarikTunai() versi override milik RekeningGiro, bukan milik Rekening biasa.

Hal terpenting dari output ini adalah saldo akhir yang bernilai negatif, yaitu Rp-300000.0. Pada rekening biasa, penarikan melebihi saldo akan ditolak, tetapi pada rekening giro penarikan hanya ditolak jika nominal melebihi saldo + batasOverdraft. Karena saldo menjadi -300.000 setelah penarikan 400.000, saldo awalnya adalah Rp100.000 dan batas overdraft minimal Rp300.000, sehingga penarikan masih diizinkan dengan memakai fasilitas overdraft.

2. Tugas/Latihan Praktikum

1. Buat Method Transfer : Di dalam class Rekening.java, tambahkan satu method bar dengan nama : public void transfer(Rekening tujuan, double nominal, String pinInput)
   a. Method ini menerima parameter polimorfik Rekening tujuan, bisa menerima Tabungan atau Giro.
   b. Logika Method:
      - Pertama, lakukan otentikasi(pinInput). Jika false, batalkan.

- Kedua, panggil this.tarikTunai(nominal). (overriding, jika this adalah Giro, ia otomatis menggunakan aturan overdraft).
- Ketiga, periksa kembalian nilai tarikTunai. Jika true, panggil tujuan.setorTunai(nominal).

2. Integrasi ke CLI Utama: Buka Main.java dan tambahkan menu 8. Transfer ke Rekening Lain.
   a. Minta pengguna memasukkan Nomor Rekening Tujuan.
   b. Cari nomor tersebut di dalam daftarRekening.
   c. Jika ketemu, minta nominal transfer dan PIN, lalu panggil akunAktif.transfer(akunTujuan, nominal, pin).

Jawaban :

1. Kode program class Rekening :

```
1. public void transfer(Rekening tujuan, double nominal, String
   pinInput) {
2.         if (!otentikasi(pinInput)) {
3.             System.out.println("Pin salah!");
4.             return;
5.         }
6.         if (this.tarikTunai(nominal)) {
7.             tujuan.setorTunai(nominal);
8.         }
9.     }
```

Kode ini adalah method transfer() pada class Rekening Pekan_5, yang memindahkan dana dari rekening pengirim (objek yang memanggil method) ke rekening tujuan. Method bertipe void dan menerima tiga parameter: tujuan (objek Rekening penerima), nominal (jumlah transfer), dan pinInput (PIN pengirim untuk verifikasi).

Baris if (!otentikasi(pinInput)) memeriksa PIN pengirim; jika salah, program menampilkan “Pin salah!” lalu return menghentikan method sehingga tidak ada dana yang berpindah. Jika PIN benar, baris if (this.tarikTunai(nominal)) menarik dana dari rekening pengirim sekaligus memanfaatkan nilai boolean yang dikembalikan: transfer hanya dilanjutkan bila penarikan berhasil (true), sedangkan bila gagal (nominal tidak valid atau

saldo tidak cukup) tidak ada setoran ke tujuan, sehingga uang tidak muncul begitu saja. Terakhir, tujuan.setorTunai(nominal) menambah saldo penerima dan mencatatnya sebagai transaksi “Kredit” di riwayatnya.

Method ini juga menerapkan polimorfisme, karena this.tarikTunai() menjalankan versi sesuai jenis objek pengirim: pada RekeningGiro transfer boleh memakai fasilitas overdraft, sedangkan pada rekening biasa dibatasi oleh saldo. Satu catatan, sisi pengirim tercatat sebagai “Debit” (atau “Debit Giro”), bukan “Transfer”, sehingga di mutasi tidak terlihat bahwa dana tersebut berasal dari proses transfer.

2. Kode program class Main :

```
1.  System.out.println("8. Transfer ke Rekening lain");
2.
3.  case 8:
4.      if (akunAktif == null) {
5.              System.out.println("Error : Anda belum memiliki akun
    aktif!");
6.      } else {
7.              System.out.print("Masukkan Nomor Rekening Tujuan : ");
8.              String nomorTujuan = input.nextLine();
9.
10.             Rekening akunTujuan = null;
11.             for (Rekening rA : rekeningAktif) {
12.                     if (rA.cocokDenganNomor(nomorTujuan)) {
13.                             akunTujuan = rA;
14.                             break;
15.                     }
16.             }
17.
18.             if (akunTujuan == null) {
19.                     System.out.println("Gagal: Rekening tujuan tidak
    ditemukan.");
20.             } else if (akunTujuan == akunAktif) {
21.                     System.out.println("Gagal: Tidak dapat transfer
    ke rekening sendiri.");
22.             } else {
23.                     System.out.print("Masukkan Nominal Transfer :
    ");
24.                     double nominal = input.nextDouble();
25.                     input.nextLine();
26.
27.                     System.out.print("Masukkan PIN : ");
28.                     String pin1 = input.nextLine();
29.
30.                     akunAktif.transfer(akunTujuan, nominal, pin1);
31.     }
```

```
32. }
33. break;
```

Kode ini adalah potongan menu 8 (Transfer ke Rekening lain) pada Main Pekan_5. Baris System.out.println("8. Transfer ke Rekening lain") menambahkan pilihan baru pada menu utama, sedangkan case 8 menjalankan proses transfer melalui method transfer() milik class Rekening. Program pertama-tama mengecek akunAktif; jika masih null, ditampilkan pesan error karena transfer membutuhkan akun pengirim yang aktif.

Jika akun aktif ada, program meminta nomor rekening tujuan dan menyimpannya di nomorTujuan. Variabel akunTujuan disiapkan bernilai null sebagai penampung hasil pencarian, lalu list rekeningAktif ditelusuri dengan for-each. Setiap rekening diperiksa dengan cocokDenganNomor(nomorTujuan), dan jika cocok, objeknya disimpan ke akunTujuan lalu break menghentikan pencarian.

Hasil pencarian divalidasi dengan struktur if-else if-else. Jika akunTujuan == null, ditampilkan pesan bahwa rekening tujuan tidak ditemukan. Jika akunTujuan == akunAktif, ditampilkan pesan bahwa transfer ke rekening sendiri tidak diperbolehkan; pengecekan ini membandingkan referensi objek sehingga dana tidak ditarik lalu disetor kembali ke rekening yang sama. Jika keduanya tidak terpenuhi, program meminta nominal dengan nextDouble(), diikuti input.nextLine() untuk membersihkan sisa enter di buffer agar input PIN tidak terlewat. Program lalu meminta PIN pengirim (disimpan di pin1) dan memanggil akunAktif.transfer(akunTujuan, nominal, pin1), yang memverifikasi PIN, menarik dana dari pengirim, lalu menyetorkannya ke rekening tujuan. Terakhir, break mengakhiri case 8 agar program kembali ke menu utama.

Output :

```
Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening lain
Pilih Menu : 8
Masukkan Nomor Rekening Tujuan : 001
Masukkan Nominal Transfer : 50000
Masukkan PIN : 000000
Tarik tunai Rp50000.0 berhasil. Sisa saldo: Rp150000.0
Setor tunai Rp50000.0 berhasil. Saldo saat ini : Rp70000.0
```

Saldo awal akun yang menjadi pengirim : Rp200.000 menjadi Rp150.000

Saldo awal akun yang menjadi tujuan: Rp20.000 menjadi Rp70.000

Jika mengirim pada nomor rekening yang akun nya sedang digunakan :

```
Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening lain
Pilih Menu : 8
Masukkan Nomor Rekening Tujuan : 000
Gagal: Tidak dapat transfer ke rekening sendiri.
```

LINK PENJELASAN KODE PROGRAM PEKAN 1 – 5:

https://youtu.be/anhEkZxvH5U